

TRAIVO — Master Implementation Guide

Version: 1.0 — 2026-05-06

Status: DEFINITIV — redo för implementation

Datamodellsbeslut: Alt B (order_concepts)

Referenskund: Kinab (~3 750 befintliga work_orders, Örebro Bostäder ~1 066 objekt)

Språk: Svenska (tekniska termer på engelska i kod)

EXECUTIVE SUMMARY

Vad är Traivo?

Traivo är ett **produktionsstyrningssystem för fältservice** — tänk ERP för företag som utför återkommande tjänster på fysiska objekt (kärltvätt, rumstvätt, besiktning, underhåll) hos kunder med hundratals eller tusentals platser.

Vad bygger vi?

Ett system som hanterar hela kedjan:

Kunddata → Prislister → Orderkoncept → Expansion → Planering → Utförande → Fakturering

Nyckelproblem som systemet löser

1. **Massexpansion** — ett orderkoncept med 6 artiklar × 582 objekt = ~2 910 individuella uppgifter per cykel
2. **Metadata-driven logik** — objekten är tunna, all styrdata (association, kvantitet, leveranstid) lever som metadata med arv genom klusterhierarkin
3. **Beroendehantering** — uppgifter kan ha sekvenskrav (hämta nyckel → tvätta kärl) med offset-tid och larm
4. **Kapacitetsplanering** — 28h produktion + 12h restid per 40h vecka, med orderlager och sökmönster
5. **Komplex fakturering** — per uppgift, månatlig eller kvartalsvis, med indexjustering och retroaktiv omräkning

Datamodellsbeslut: Alt B

Ny **order_concepts** -tabell separerad från **work_orders**. Befintliga 3 750+ work_orders opåverkade. Full bakåtkompatibilitet med mobile (Go), VRP, Fortnox och import.

1. KÄRNKONCEPT — Mats Mentala Modell

De 5 kärnkoncepten

1.1 Objekt (minimal entitet + metadata-subsystem)

Objektet är **extremt tunt** — bara ID, namn och hierarki. All verklig information lever som metadata.

```
ServiceObject = {
  id, name, external_number, parent_id, customer_id
}
```

Allt annat — objekttyp, association, kvantitet, adress, GPS, kontaktinfo, foton, leveransvillkor — är metadata-poster ("post-it-lappar") som hakas på objektet.

1.2 Metadata (EAV med arv och nivåås)

Metadata är ett **egget subsystem** med sofistikerad styrlogik:

- **Arv** — metadata flödar automatiskt nedåt i klusterhierarkin (Område → Plats → Objekt)
- **Nivåås** — stoppar arv, metadata gäller enbart det låsta objektet
- **Versionering** — gamla värden arkiveras automatiskt med spårbarhet (vem, när, från/till)
- **Ägarskap** — skapandemetod (manuell/inläst/automatisk/vid utförande), ändringslås
- **Format** — text, numeriskt, bild, datum-range, GPS, boolean

1.3 Artiklar (produktionsinstruktioner med 33 fält)

En artikel är **inte bara en prisrad** — det är en komplett produktionsregel:

- **Affärslager** — artikelnummer, kostnad, pris, enhet, fakturabeskrivning
- **Produktionslager** — intern instruktion, produktionstid, utförandetyp, QA-länk, tilldelning
- **Datalager** — vilken metadata den läser från objektet och skriver tillbaka
- **Strukturlager** — kan vara BOM (checklista med underartiklar)
- **Beroenden** — kan kräva att andra artiklar utförs först

1.4 OrderConcept (mallar för återkommande jobb)

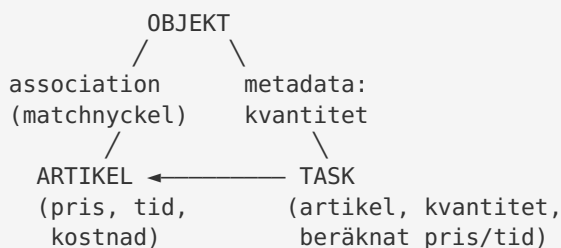
Samlar artiklar till ett uppdrag. Lever permanent som mall för periodicitet/regenerering.

```
OrderConcept "ÖB0 Standarduppdrag" = {
  K100 (Kärlltvätt), R100 (Rumstvätt), F100 (Foto),
  A100 (Avisering), N100 (Hämta nyckel), AV100 (Avvikelse)
}
```

1.5 Task (expanderad work_order från concept)

Konkret arbetsrad med fryst kvantitet/pris/tid. En per artikel × objekt.

Triangulär logik



Flödet:

1. Objekt har metadata Association = "RBK Standard" och ANT = 9
2. Artikel K100 har association = "RBK Standard", pris = 150 kr, tid = 3 min
3. Expansion skapar Task: 9 kärll × 150 kr = 1 350 kr, 9 × 3 min = 27 min

2. DEN RÖDA TRÅDEN — Affärsflödet

Komplett flöde från start till slut

1. KUNDDATA IMPORTERAS
Excel → service_objects + object_metadata
(objekt + association + kvantitet + adress + leveransvillkor)
2. PRISLISTA/ARTIKLAR DEFINIERAS
articles (33 fält) + article_components (BOM)
+ article_dependencies (sekvenskrav)
3. ORDERKONCEPT SKAPAS
Välj artiklar + association-matchning + periodicitet
billing_mode: per_task / monthly / quarterly
4. EXPANSION: Concept → Tasks
expandOrderConcept() → metadata-arv, mängdbegränsning,
BOM → sub-tasks, beroenden → task dependencies
5. PLANERING
Orderlager → Grovplanering (vecka) → Finplanering (rutt)
Sökmönster, kapacitetssummering, 28h/12h-mål
6. UTFÖRANDE
Mobil-app (Go): checklista, data read/write, GPS,
foto, signatur, offline-support
7. KVALITETSKONTROLL & FAKTURERING
QA OK → READY_TO_INVOICE → Fortnox-export → INVOICED
Månadsfakturerering med indexjustering

Konkret exempel: Örebro Bostäder

1. Import: 582 adresser × 2 objekttyper = 1 164 objekt
2. Prislista: 6 artiklar (K100, R100, F100, A100, N100, AV100)
3. OrderConcept: "ÖBO Standarduppdrag" med alla 6 artiklar
4. Expansion:

K100	×	582 objekt (RBK Standard)	=	582 tasks
R100	×	582 objekt (Rum m2)	=	582 tasks
F100	×	582 objekt (single)	=	582 tasks
A100	×	582 objekt (offset: 2h)	=	582 tasks
N100	×	582 objekt (offset: 40h)	=	582 tasks
- Totalt: ~2 910 tasks per uppdragscykel
5. Planering: Fördela 2 910 tasks över veckor/team
6. Utförande: Team kör rutt, checkar av, rapporterar
7. Fakturerering: Per task eller månatlig

3. KOMPLETT DATAMODELL (#356 Alt B)

3.1 service_objects (minimal)

```
CREATE TABLE service_objects (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id   UUID NOT NULL REFERENCES tenants(id),  
  customer_id UUID NOT NULL REFERENCES customers(id),  
  parent_id   UUID REFERENCES service_objects(id),  
  
  name        TEXT NOT NULL,          -- "Brickebacken Standardkärll"  
  external_number TEXT,              -- Objektnummer från kundsystem  
  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW(),  
  deleted_at  TIMESTAMPTZ  
);  
  
CREATE INDEX idx_so_parent ON service_objects(parent_id);  
CREATE INDEX idx_so_customer ON service_objects(customer_id);  
CREATE INDEX idx_so_tenant ON service_objects(tenant_id);
```

3.2 object_metadata (EAV med arv)

```

CREATE TABLE object_metadata (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id),
  object_id   UUID NOT NULL REFERENCES service_objects(id),

  -- Identitet
  metadata_type TEXT NOT NULL,          -- "Association", "Önskad leveransperi-
od", etc.
  designation   TEXT NOT NULL,          -- Kortkod: LEV, OMR, ANT, KOD, KUND,
etc.

  -- Innehåll
  value         JSONB NOT NULL,         -- Flexibelt: text, nummer, bild-URL,
etc.
  value_format  TEXT,                   -- "text", "numeric", "image",
"date_range",
                                         -- "boolean", "gps"

  -- Arv och nivåås
  level_locked  BOOLEAN DEFAULT FALSE,   -- TRUE = gäller enbart detta objekt
inheritable    BOOLEAN DEFAULT TRUE,    -- FALSE = ärvs INTE nedåt
inherited_from UUID REFERENCES service_objects(id),
                                         -- NULL = direkt satt, UUID = ärvd källa

  -- Ägarskap
  created_by    UUID,
  creation_method TEXT NOT NULL DEFAULT 'manual',
                                         -- 'manual', 'import', 'automatic', 'at_execution', 'external'
  change_lock   TEXT DEFAULT 'all',
                                         -- 'no_change', 'admin_only', 'executor_allowed', 'all'

  -- Versionering
  version       INT NOT NULL DEFAULT 1,
  valid_from    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  valid_to      TIMESTAMPTZ,            -- NULL = aktiv

  -- Status
  status        TEXT NOT NULL DEFAULT 'active'
               CHECK (status IN ('active', 'archived')),

  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  archived_at   TIMESTAMPTZ,
  archived_by   UUID
);

CREATE INDEX idx_om_object ON object_metadata(object_id);
CREATE INDEX idx_om_designation ON object_metadata(designation);
CREATE INDEX idx_om_active ON object_metadata(object_id, status) WHERE status = 'active';
CREATE INDEX idx_om_tenant ON object_metadata(tenant_id);

```

3.3 articles (33 fält)

```

CREATE TABLE articles (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         UUID NOT NULL REFERENCES tenants(id),
  customer_id       UUID REFERENCES customers(id), -- NULL = global

  -- Grunddata
  article_number    TEXT NOT NULL,                -- "K100", "RBK100"
  alternative_number TEXT,                        -- Kundens/leverantörens nummer
  name              TEXT NOT NULL,                -- "Kärlltvätt standard"
  association        TEXT,                        -- "RBK Standard", "RBK1", NULL
(AV100)

  -- Prissättning
  cost              DECIMAL(10,2) DEFAULT 0, -- Intern kostnad per enhet
  price             DECIMAL(10,2) DEFAULT 0, -- Kundpris per enhet
  unit              TEXT DEFAULT 'ST',         -- "ST", "m2", "tim"

  -- Fakturering
  invoice_description TEXT,                      -- "Tvätt 100-600 liter kärl het-
vatten"
  billing_mode        TEXT DEFAULT 'per_unit',
  CHECK (billing_mode IN ('per_unit', 'fixed', 'monthly')),

  -- Utförande
  execution_type      TEXT,
-- 'vehicle_team', 'admin_phone', 'service_welding'
  internal_instructions TEXT,                    -- "Standard 3 min"
  qa_checklist_url    TEXT,                      -- "www.kinab.se/RBK"

  -- Data read/write
  data_read_keys      TEXT[],                    -- '{ANT}', '{KOD}' – metadata
att läsa
  data_write_keys     TEXT[],                    -- '{ANT}', '{KOD}' – metadata
att skriva
  data_format          TEXT,                      -- 'numeric', 'text', 'image',
'boolean'

  -- Mängd
  quantity_limit_mode TEXT NOT NULL DEFAULT 'multiply'
  CHECK (quantity_limit_mode IN (
    'multiply', -- Multiplicera med objektantal (K100)
    'single',   -- En per adress/position (TILG100)
    'configurable' -- Bestäms per orderkoncept-rad
  )),

  -- QA
  requires_photo      BOOLEAN DEFAULT FALSE,
  requires_signature  BOOLEAN DEFAULT FALSE,

  -- Struktur
  is_structure         BOOLEAN DEFAULT FALSE, -- BOM: expanderar till check-
lista
  is_component         BOOLEAN DEFAULT FALSE, -- Är del av en strukturartikel

  -- Logistik
  production_time      DECIMAL(10,2) DEFAULT 0, -- Minuter per enhet
  offset_hours         DECIMAL(10,2) DEFAULT 0, -- Timmar FÖRE huvuduppdrag
  delivery_time         TEXT,                    -- "Ställ/resa", "20+ställ"
  storage_location      TEXT,                    -- Lagerplats för fysisk artikel

  -- Periodicitet (defaults)
  default_periodicity  TEXT,                      -- 'weekly', 'monthly',

```

```

'quarterly'
  season_restriction      JSONB,                -- {"start": "0201", "end":
"0430"}

  -- Typ
  article_type            TEXT DEFAULT 'service'
  CHECK (article_type IN ('service', 'product', 'inspection', 'notification', 'd
eviation')),

  -- Tilldelning
  assignment_scope        TEXT,                -- 'district', 'team', 'individu-
al'

  -- Ersättning
  replaces_article_id     UUID REFERENCES articles(id),

  created_at              TIMESTAMPTZ DEFAULT NOW(),
  updated_at              TIMESTAMPTZ DEFAULT NOW(),
  deleted_at              TIMESTAMPTZ

);

CREATE INDEX idx_articles_association ON articles(association);
CREATE INDEX idx_articles_tenant ON articles(tenant_id);
CREATE UNIQUE INDEX idx_articles_number_tenant
  ON articles(article_number, tenant_id) WHERE deleted_at IS NULL;

```

3.4 article_components (BOM)

```

CREATE TABLE article_components (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  parent_article_id UUID NOT NULL REFERENCES articles(id) ON DELETE CASCADE,
  child_article_id  UUID NOT NULL REFERENCES articles(id),
  sort_order        INT NOT NULL DEFAULT 0,
  is_mandatory      BOOLEAN DEFAULT TRUE,

  UNIQUE(parent_article_id, child_article_id)
);

CREATE INDEX idx_ac_parent ON article_components(parent_article_id);

```

Exempeldata:

```

-- TILG100 "Kodlås" med underartiklar
INSERT INTO article_components VALUES
  (gen_random_uuid(), '{TILG100_id}', '{TILG201_id}', 1, TRUE),
  (gen_random_uuid(), '{TILG100_id}', '{TILG202_id}', 2, TRUE);

-- SA-K100 "Kärltvätt komplett" (från session)
-- K101: Tvätt invändig + utvändig
-- K102: Applicering fraktionsdekal
-- K103: Torkning handtag
-- K104: Applicering efterbehandling
-- K105: Foto efter utfört uppdrag

```


3.5 article_dependencies

```
CREATE TABLE article_dependencies (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  article_id  UUID NOT NULL REFERENCES articles(id) ON DELETE CASCADE,
  depends_on_article_id UUID NOT NULL REFERENCES articles(id),
  dependency_type TEXT NOT NULL DEFAULT 'must_complete_first'
  CHECK (dependency_type IN (
    'must_complete_first', -- Blockerande
    'should_complete_first', -- Mjukt beroende
    'must_start_first' -- Beroende måste ha startats
  )),
  UNIQUE(article_id, depends_on_article_id)
);
```

Exempeldata:

```
-- Kärntvätt beror på kodkontroll och nyckelhämtning
INSERT INTO article_dependencies VALUES
(gen_random_uuid(), '{RBK100_id}', '{TILG100_id}', 'must_complete_first'),
(gen_random_uuid(), '{RBK100_id}', '{N100_id}', 'must_complete_first');
```

3.6 order_concepts

```

CREATE TABLE order_concepts (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id),
  customer_id UUID NOT NULL REFERENCES customers(id),

  title       TEXT NOT NULL,           -- "ÖBO Standarduppdrag"
  description  TEXT,
  concept_status TEXT DEFAULT 'draft'
    CHECK (concept_status IN ('draft', 'active', 'paused', 'completed',
                              'archived')),

  -- Periodicitet
  is_recurring    BOOLEAN DEFAULT FALSE,
  recurrence_interval INTEGER,         -- Antal
  recurrence_unit  TEXT,               -- 'days', 'weeks', 'months'
  tolerance_days   INTEGER DEFAULT 0,  -- ±dagar tolerans
  next_generation_date TIMESTAMPTZ,
  last_generated_at TIMESTAMPTZ,

  -- Fakturering
  billing_mode     TEXT DEFAULT 'per_task'
    CHECK (billing_mode IN ('per_task', 'monthly', 'quarterly')),
  annual_planned_value DECIMAL(12,2), -- Årsbelopp (monthly: /12, quarterly: /

4)

  -- Leveranstid
  desired_delivery_start DATE,
  desired_delivery_end   DATE,
  season_name            TEXT,           -- "Vårtvätt", "Hösttvätt"

  -- Metadata
  external_reference TEXT,
  notes              TEXT,
  metadata           JSONB,

  created_by         UUID,
  created_at         TIMESTAMPTZ DEFAULT NOW(),
  updated_at         TIMESTAMPTZ DEFAULT NOW(),
  deleted_at         TIMESTAMPTZ
);

CREATE INDEX idx_oc_customer ON order_concepts(customer_id);
CREATE INDEX idx_oc_status ON order_concepts(concept_status);
CREATE INDEX idx_oc_next_gen ON order_concepts(next_generation_date)
  WHERE is_recurring = TRUE;

```

3.7 order_concept_lines

```

CREATE TABLE order_concept_lines (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         UUID NOT NULL REFERENCES tenants(id),
  order_concept_id  UUID NOT NULL REFERENCES order_concepts(id) ON DELETE CASCADE,
  article_id        UUID NOT NULL REFERENCES articles(id),

  default_quantity  INTEGER DEFAULT 1,
  quantity_mode_override TEXT,           -- Override av artikelns quantity_limit_mode
  association_filter TEXT,               -- Vilken association att matcha
  is_optional       BOOLEAN DEFAULT FALSE,
  sort_order        INTEGER DEFAULT 0,
  notes             TEXT,

  created_at        TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_ocl_concept ON order_concept_lines(order_concept_id);

```

3.8 work_orders (tasks med status)

```

CREATE TABLE work_orders (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         UUID NOT NULL REFERENCES tenants(id),

  -- Koppling uppåt
  order_concept_id  UUID REFERENCES order_concepts(id),
  concept_line_id   UUID REFERENCES order_concept_lines(id),
  service_object_id UUID NOT NULL REFERENCES service_objects(id),
  article_id        UUID NOT NULL REFERENCES articles(id),

  -- Fryst data vid skapande (snapshot)
  frozen_quantity   DECIMAL(10,2) NOT NULL,
  frozen_unit_price DECIMAL(10,2) NOT NULL,
  frozen_unit_cost  DECIMAL(10,2) NOT NULL,
  frozen_exec_time  DECIMAL(10,2) NOT NULL, -- Minuter per enhet
  frozen_unit       TEXT DEFAULT 'ST',

  -- Beräknade värden
  total_price       DECIMAL(10,2) GENERATED ALWAYS AS (
    frozen_quantity * frozen_unit_price) STORED,
  total_cost        DECIMAL(10,2) GENERATED ALWAYS AS (
    frozen_quantity * frozen_unit_cost) STORED,
  total_time        DECIMAL(10,2) GENERATED ALWAYS AS (
    frozen_quantity * frozen_exec_time) STORED,
  margin            DECIMAL(10,2) GENERATED ALWAYS AS (
    frozen_quantity * (frozen_unit_price - frozen_unit_cost)) STORED,

  -- Status (8 steg)
  status            TEXT NOT NULL DEFAULT 'unplanned'
    CHECK (status IN (
      'unplanned', 'rough_planned', 'fine_planned',
      'en_route', 'on_site',
      'completed', 'failed',
      'ready_to_invoice', 'invoiced', 'cancelled'
    )),

  -- Tidsstämplar per status
  rough_planned_at  TIMESTAMPTZ,
  rough_planned_week INTEGER,           -- Veckonummer
  fine_planned_at   TIMESTAMPTZ,
  scheduled_date    DATE,
  scheduled_time    TIME,
  en_route_at       TIMESTAMPTZ,
  on_site_at        TIMESTAMPTZ,
  completed_at      TIMESTAMPTZ,
  invoiced_at       TIMESTAMPTZ,

  -- Offset-schemaläggning
  offset_scheduled_at TIMESTAMPTZ,     -- scheduled_at - offset_hours

  -- BOM-koppling (strukturartikel → checklista)
  parent_work_order_id UUID REFERENCES work_orders(id),
  checklist_order      INT,

  -- Utförande
  execution_code      TEXT,             -- Cachat från artikel
  team_id             UUID REFERENCES teams(id),
  resource_id         UUID,

  -- Resultat
  actual_quantity     DECIMAL(10,2),    -- Faktiskt utfört antal
  not_completed_reason TEXT,

```

```

    result_data          JSONB,          -- Utförarens data (foto, kvittens)

    -- Tidsfönster
    preferred_windows    JSONB,          -- Ärvda leveransvillkor

    -- GPS
    checkin_lat          DECIMAL(10,7),
    checkin_lng          DECIMAL(10,7),
    checkout_lat         DECIMAL(10,7),
    checkout_lng         DECIMAL(10,7),

    -- Metadata-snapshot
    metadata_snapshot    JSONB,          -- Data read vid start

    notes                TEXT,
    created_at           TIMESTAMPTZ DEFAULT NOW(),
    updated_at           TIMESTAMPTZ DEFAULT NOW(),
    deleted_at           TIMESTAMPTZ
);

CREATE INDEX idx_wo_concept ON work_orders(order_concept_id);
CREATE INDEX idx_wo_object ON work_orders(service_object_id);
CREATE INDEX idx_wo_status ON work_orders(status);
CREATE INDEX idx_wo_scheduled ON work_orders(scheduled_date);
CREATE INDEX idx_wo_team ON work_orders(team_id);
CREATE INDEX idx_wo_parent ON work_orders(parent_work_order_id);

```

3.9 work_order_dependencies

```

CREATE TABLE work_order_dependencies (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    work_order_id     UUID NOT NULL REFERENCES work_orders(id) ON DELETE CAS-
CADE,
    depends_on_work_order_id UUID NOT NULL REFERENCES work_orders(id),
    dependency_type    TEXT NOT NULL DEFAULT 'must_complete_first',
    is_satisfied       BOOLEAN DEFAULT FALSE,
    satisfied_at       TIMESTAMPTZ,

    UNIQUE(work_order_id, depends_on_work_order_id)
);

CREATE INDEX idx_wod_wo ON work_order_dependencies(work_order_id);
CREATE INDEX idx_wod_depends ON work_order_dependencies(depends_on_work_order_id);

```

3.10 teams (med kapacitetsmål)

```
CREATE TABLE teams (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         UUID NOT NULL REFERENCES tenants(id),
  name              TEXT NOT NULL,

  -- Kapacitetsmål (28/12-regel)
  production_hours_target DECIMAL(5,2) DEFAULT 28.0,
  travel_hours_target     DECIMAL(5,2) DEFAULT 12.0,
  total_hours_week        DECIMAL(5,2) DEFAULT 40.0,

  -- Utförarens senaste position
  last_position_lat  DECIMAL(10,7),
  last_position_lng  DECIMAL(10,7),
  last_position_at   TIMESTAMPTZ,

  -- Vila
  rest_type          TEXT CHECK (rest_type IN ('none', 'daily_rest', 'weekly_rest'))
),
  rest_location      TEXT,
  rest_until         TIMESTAMPTZ,

  -- Utförandekoder (execution codes teamet kan hantera)
  execution_codes     TEXT[],          -- '{vehicle_team, admin_phone}'

  created_at         TIMESTAMPTZ DEFAULT NOW(),
  updated_at         TIMESTAMPTZ DEFAULT NOW()
);
```

3.11 planner_search_filters

```
CREATE TABLE planner_search_filters (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         UUID NOT NULL REFERENCES tenants(id),
  name              TEXT NOT NULL,
  created_by        UUID NOT NULL,
  is_static          BOOLEAN DEFAULT TRUE,    -- FALSE = tillfälligt

  -- Filterkriterier
  filter_criteria    JSONB NOT NULL,
  -- Exempel:
  -- {
  --   "execution_types": ["vehicle_team"],
  --   "postal_codes": ["85230", "85231"],
  --   "geographic_area": "Sundsvall",
  --   "article_associations": ["RBK Standard"],
  --   "status": ["unplanned", "rough_planned"],
  --   "date_range": {"from": "2026-05-01", "to": "2026-05-31"}
  -- }

  team_id           UUID REFERENCES teams(id),
  created_at        TIMESTAMPTZ DEFAULT NOW()
);
```

3.12 article_translations

```
CREATE TABLE article_translations (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  article_id  UUID NOT NULL REFERENCES articles(id) ON DELETE CASCADE,  
  language_code TEXT NOT NULL,          -- 'sv', 'en', 'no'  
  invoice_description TEXT,  
  internal_instructions TEXT,  
  name        TEXT,  
  
  UNIQUE(article_id, language_code)  
);
```


3.13 Viktiga SQL-funktioner

get_effective_metadata()

```

CREATE OR REPLACE FUNCTION get_effective_metadata(
    p_object_id UUID,
    p_designation TEXT DEFAULT NULL
)
RETURNS TABLE (
    metadata_id      UUID,
    object_id        UUID,
    designation       TEXT,
    value            JSONB,
    value_format      TEXT,
    level_locked      BOOLEAN,
    source_object_id UUID,
    is_inherited      BOOLEAN
)
LANGUAGE sql STABLE AS $$
    WITH RECURSIVE ancestors AS (
        SELECT id, parent_id, 0 AS depth
        FROM service_objects WHERE id = p_object_id

        UNION ALL

        SELECT so.id, so.parent_id, a.depth + 1
        FROM service_objects so
        JOIN ancestors a ON so.id = a.parent_id
    ),
    candidate_metadata AS (
        SELECT
            om.id AS metadata_id,
            om.object_id,
            om.designation,
            om.value,
            om.value_format,
            om.level_locked,
            om.object_id AS source_object_id,
            (om.object_id != p_object_id) AS is_inherited,
            a.depth,
            ROW_NUMBER() OVER (
                PARTITION BY om.designation
                ORDER BY a.depth ASC
            ) AS rn
        FROM object_metadata om
        JOIN ancestors a ON om.object_id = a.id
        WHERE om.status = 'active'
            AND (p_designation IS NULL OR om.designation = p_designation)
            AND (
                om.level_locked = FALSE
                OR om.object_id = p_object_id
            )
    )
    SELECT metadata_id, object_id, designation, value, value_format,
           level_locked, source_object_id, is_inherited
    FROM candidate_metadata
    WHERE rn = 1
$$;

```

expand_order_concept() (se sektion 6 för full TypeScript-version)
calculate_capacity_summary()

```
CREATE OR REPLACE FUNCTION calculate_capacity_summary(
  p_team_id UUID,
  p_week_start DATE,
  p_week_end DATE
)
RETURNS TABLE (
  total_production_hours DECIMAL,
  total_travel_hours     DECIMAL,
  task_count             INTEGER,
  capacity_utilization   DECIMAL
)
LANGUAGE sql STABLE AS $$
  SELECT
    COALESCE(SUM(wo.total_time) / 60.0, 0) AS total_production_hours,
    0.0 AS total_travel_hours, -- Beräknas separat via VRP
    COUNT(*)::INTEGER AS task_count,
    CASE
      WHEN t.production_hours_target > 0
      THEN (COALESCE(SUM(wo.total_time) / 60.0, 0) / t.production_hours_target)
* 100
      ELSE 0
    END AS capacity_utilization
  FROM work_orders wo
  JOIN teams t ON t.id = p_team_id
  WHERE wo.team_id = p_team_id
    AND wo.scheduled_date BETWEEN p_week_start AND p_week_end
    AND wo.status NOT IN ('cancelled', 'invoiced')
  GROUP BY t.production_hours_target
$$;
```

4. ARTIKELLOGIK — 33 Fält Förklarar

#	Fält	DB-kolumn	Typ	Beskrivning	Exempel
1	Art nummer	art- icle_number	TEXT	Unik artikelkod	RBK100
2	Alternativt nummer	alternat- ive_number	TEXT	Kundens/ lever- antörens nummer	K-4521
3	Underordnad art	via art- icle_compo- nents	FK	BOM-relation	TILG201 un- der TILG100
4	Namn	name	TEXT	Artikels namn	Kärلتvätt standard
5	Beskrivning	in- voice_descri- ption	TEXT	Faktu- rabeskrivning (kunden ser)	Tvätt 100-600 liter kärل hetvatten
6	Ut- förandebeskrivning	intern- al_instructi- ons	TEXT	Intern in- struktion (ut- föraren ser)	Standard 3 min
7	Länk info	qa_checklist _url	TEXT	QA/instruk- tionslänk	www.kinab.se /RBK
8	Struktur- artikel	is_structure	BOOL	BOM-flagga	TRUE → ex- panderar
9	Typ	article_type	TEXT	Artikelkat- egori	service , product , deviation
10	Association	association	TEXT	Matchnyckel mot objekt	RBK1 , KOD1
11	Begränsning	quant- ity_limit_mo- de	TEXT	Mängdlogik	multiply , single , configurable
12	Enhet	unit	TEXT	Prisenhet	ST , m2 , tim
13	Kostnad	cost	DECIMAL	Intern kost- nad/enhet	150

#	Fält	DB-kolumn	Typ	Beskrivning	Exempel
14	Försäljningspris	price	DECIMAL	Kundpris/enhet	250
15	Leveranstid	delivery_time	TEXT	Leveranstidsbeskrivning	Ställ/resa , 20+ställ
16	Lagerplats	storage_location	TEXT	Fysisk lagerplats	Bjästa, hylla 3
17	Produktionstid	production_time	DECIMAL	Minuter/enhet	3
18	Hämta data	data_read_keys	TEXT[]	Metadata att läsa	{ANT}
19	Metadata-typ (read)	(del av data_read_keys)	TEXT	Vilken beteckning	ANT , KOD
20	Lämna data	data_write_keys	TEXT[]	Metadata att skriva	{ANT}
21	Metadata-typ (write)	(del av data_write_keys)	TEXT	Vilken beteckning	ANT , KOD
22	Format	data_format	TEXT	Dataformat	numeric , text , image
23	Utförande	execution_type	TEXT	Resurstyp	vehicle_team , admin_phone
24-31	Statuskolumner	via work_orders.status	ENUM	8-steps flöde	Se sektion 7
32	Tilldelning	assignment_scope	TEXT	Tilldelningsnivå	district , team
33	Tilldelning detalj	(team_id vid tilldelning)	UUID	Specifik tilldelning	Team "Syd"

quantity_limit_mode — tre scenarion

MULTIPLY (default):
 K100 + objekt med ANT=40 → 40 enheter i en task
 40 kärl × 150 kr = 6 000 kr ✓

SINGLE:
 TILG100 + objekt med ANT=40 → 1 enhet (en per adress)
 1 × 100 kr = 100 kr ✓

CONFIGURABLE:
 F100 + kund bestämmer → antingen 1 foto eller 40 foton
 Bestäms per OrderConcept-rad ✓

5. METADATALOGIK — Arv och Nivålå

Hur arv fungerar (Område → Plats → Objekt)

```

Axfood Sverige (metadata: KUND = "Axfood Sverige")
├── Distrikt Sydöst
│   ├── metadata: LEV = "0201-0430, Torsdag EM" ← EJ nivålåst → ärvs nedåt
│   ├── Butik Malmö (ärver LEV "Torsdag EM")
│   │   ├── Soprum Södra
│   │   │   ├── Matavfallskärl x10 → ärver KUND + LEV
│   │   │   └── Papperskärl x5 → ärver KUND + LEV
│   │   └── Soprum Norra
│   └── Butik Lund
│       ├── metadata: LEV = "Fredag FM" ← NIVÅLÅST → override!
│       └── 8 matavfallskärl → FÅR "Fredag FM" (inte Torsdag EM)
└── Distrikt Mälardalen
    ├── metadata: LEV = "0501-0930, Onsdag FM"
    └── ...
  
```

Sju arvregler

Regel	Beskrivning
R-META-1	Metadata utan nivålås ärvs automatiskt nedåt i hela klusterträdet
R-META-2	Nivålås stoppar arv — metadata gäller enbart det låsta objektet
R-META-3	Ärvt metadata visas i UI med länk till ursprungsnivå
R-META-4	Override: ny metadata med nivålås på lägre nivå bryter arvkedjan
R-META-5	Motstridiga platsdata → systemet varnar (blockerar inte)
R-META-6	Gamla metadata arkiveras automatiskt vid uppdatering
R-META-7	Arkivering bevarar: vem, när, tidigare värde

level_locked och inheritable

```
-- level_locked = TRUE: "Denna metadata gäller BARA detta objekt"
-- level_locked = FALSE: "Denna metadata ärvs nedåt i trädet"
-- inheritable = FALSE: "Ärvs aldrig, oavsett level_locked"

-- Exempelscenario: Association
-- Association är ALLTID nivålåst (gäller exakt detta objekt)
INSERT INTO object_metadata (object_id, designation, value, level_locked)
VALUES ('{behållare_id}', 'Association', '"Matavfallskärl 125 liter"', TRUE);

-- Leveransperiod ärvs normalt (sätts på distrikt, flyter ned)
INSERT INTO object_metadata (object_id, designation, value, level_locked)
VALUES ('{distrikt_id}', 'LEV', '"0201-0430, Torsdag EM"', FALSE);
```

Versionering

```
Metadata "ANT" på Behållare 34243:
v1: 8   (2026-01-03, Anna, Inläst)   → status='archived', valid_to=2026-05-28
v2: 10  (2026-05-28, Johan, Utförande) → status='active',  valid_to=NULL
```

Vid uppdatering:

1. Arkivera befintlig post (status='archived' , valid_to=NOW())
2. Skapa ny post (version = old.version + 1 , valid_from=NOW())
3. Logga vem och hur

Komplett get_effective_metadata() med exempel

```
-- Hämta alla aktiva metadata för objekt (inkl ärvda)
SELECT * FROM get_effective_metadata('{matavfallskärl_id}');

-- Resultat:
-- designation | value                                | is_inherited | source
-- KUND        | "Axfood Sverige"                    | TRUE         | {axfood_id}
-- LEV         | "0201-0430, Torsdag EM"             | TRUE         | {distrikt_id}
-- Association | "Matavfallskärl 125 liter"          | FALSE        | {matavfallskärl_id}
-- ANT         | 10                                  | FALSE        | {matavfallskärl_id}
-- Adress      | "Patronvägen 1 Tunadal"             | TRUE         | {butik_id}

-- Hämta specifik metadata
SELECT * FROM get_effective_metadata('{matavfallskärl_id}', 'ANT');
-- → value: 10, is_inherited: FALSE
```

6. EXPANSION — OrderConcept → Tasks

Komplett TypeScript-kod

```

interface ExpandedTask {
  serviceObjectId: string;
  articleId: string;
  conceptLineId: string;
  frozenQuantity: number;
  frozenUnitPrice: number;
  frozenUnitCost: number;
  frozenExecTime: number;
  frozenUnit: string;
  executionCode: string | null;
  offsetScheduledAt: Date | null;
  parentWorkOrderId: string | null;
  checklistOrder: number | null;
  metadataSnapshot: Record<string, any>;
  preferredWindows: any | null;
}

interface TaskDependency {
  workOrderId: string; // placeholder – sätt efter insert
  dependsOnWorkOrderId: string;
  dependencyType: string;
}

async function expandOrderConcept(
  conceptId: string,
  baseScheduleDate: Date
): Promise<{ tasks: ExpandedTask[]; dependencies: TaskDependency[] }> {

  const concept = await db.getOrderConcept(conceptId);
  const lines = await db.getConceptLines(conceptId);
  const objects = await db.getServiceObjects(concept.customerId);

  const tasks: ExpandedTask[] = [];
  const dependencies: TaskDependency[] = [];

  // Spåra tasks per (article, object) för dependency-koppling
  const taskMap = new Map<string, ExpandedTask>();

  for (const line of lines) {
    const article = await db.getArticle(line.articleId);

    // — STEG 1: ASSOCIATION-MATCHNING VIA METADATA —
    if (!article.association) continue; // AV100: genereras ej automatiskt

    const matchingObjects: Array<{
      obj: ServiceObject;
      meta: EffectiveMetadata[];
    }> = [];

    for (const obj of objects) {
      const meta = await db.getEffectiveMetadata(obj.id);
      const associations = meta
        .filter(m => m.designation === 'Association')
        .map(m => typeof m.value === 'string' ? m.value : m.value);

      if (associations.includes(article.association)) {
        matchingObjects.push({ obj, meta });
      }
    }

    for (const { obj, meta } of matchingObjects) {

```

```

// — STEG 2: QUANTITY_LIMIT_MODE —
const mode = line.quantityModeOverride || article.quantityLimitMode;
let quantity: number;

switch (mode) {
  case 'multiply': {
    const antMeta = meta.find(m => m.designation === 'ANT');
    quantity = antMeta ? parseFloat(antMeta.value as string) : 1;
    break;
  }
  case 'single':
    quantity = 1;
    break;
  case 'configurable':
    quantity = line.defaultQuantity || 1;
    break;
  default:
    quantity = 1;
}

// — STEG 3: DATA SNAPSHOT (read keys) —
const snapshot: Record<string, any> = {};
if (article.dataReadKeys?.length) {
  for (const key of article.dataReadKeys) {
    const m = meta.find(m => m.designation === key);
    if (m) snapshot[key] = m.value;
  }
}

// — STEG 4: OFFSET-TID —
let offsetScheduledAt: Date | null = null;
if (article.offsetHours > 0) {
  offsetScheduledAt = new Date(
    baseScheduleDate.getTime() - article.offsetHours * 60 * 60 * 1000
  );
}

// — STEG 5: LEVERANSVILLKOR (från metadata) —
const levMeta = meta.filter(m => m.designation.startsWith('LEV'));
const preferredWindows = levMeta.length > 0
  ? levMeta.map(m => ({ designation: m.designation, value: m.value }))
  : null;

// — STEG 6: STRUKTURARTIKEL (BOM) → SUB-TASKS —
if (article.isStructure) {
  // Skapa huvud-task
  const parentTask: ExpandedTask = {
    serviceObjectId: obj.id,
    articleId: article.id,
    conceptLineId: line.id,
    frozenQuantity: quantity,
    frozenUnitPrice: article.price,
    frozenUnitCost: article.cost,
    frozenExecTime: article.productionTime,
    frozenUnit: article.unit,
    executionCode: article.executionType,
    offsetScheduledAt,
    parentWorkOrderId: null,
    checklistOrder: null,
    metadataSnapshot: snapshot,
    preferredWindows,
  };
  tasks.push(parentTask);
}

```

```

taskMap.set(`${article.id}:${obj.id}`, parentTask);

// Expander BOM → checklista sub-tasks
const components = await db.getArticleComponents(article.id);
for (let i = 0; i < components.length; i++) {
  const comp = components[i];
  const childArticle = await db.getArticle(comp.childArticleId);

  // Child snapshot
  const childSnapshot: Record<string, any> = {};
  if (childArticle.dataReadKeys?.length) {
    for (const key of childArticle.dataReadKeys) {
      const m = meta.find(m => m.designation === key);
      if (m) childSnapshot[key] = m.value;
    }
  }

  const childTask: ExpandedTask = {
    serviceObjectId: obj.id,
    articleId: childArticle.id,
    conceptLineId: line.id,
    frozenQuantity: quantity,
    frozenUnitPrice: childArticle.price,
    frozenUnitCost: childArticle.cost,
    frozenExecTime: childArticle.productionTime,
    frozenUnit: childArticle.unit,
    executionCode: childArticle.executionType,
    offsetScheduledAt: childArticle.offsetHours > 0
      ? new Date(baseScheduleDate.getTime() - childArticle.offsetHours * 36000
00)
      : null,
    parentWorkOrderId: '__PARENT_PLACEHOLDER__', // Sätts efter insert
    checklistOrder: i + 1,
    metadataSnapshot: childSnapshot,
    preferredWindows,
  };
  tasks.push(childTask);
  taskMap.set(`${childArticle.id}:${obj.id}`, childTask);
}
} else {
  // Vanlig artikel
  const task: ExpandedTask = {
    serviceObjectId: obj.id,
    articleId: article.id,
    conceptLineId: line.id,
    frozenQuantity: quantity,
    frozenUnitPrice: article.price,
    frozenUnitCost: article.cost,
    frozenExecTime: article.productionTime,
    frozenUnit: article.unit,
    executionCode: article.executionType,
    offsetScheduledAt,
    parentWorkOrderId: null,
    checklistOrder: null,
    metadataSnapshot: snapshot,
    preferredWindows,
  };
  tasks.push(task);
  taskMap.set(`${article.id}:${obj.id}`, task);
}
}
}

```

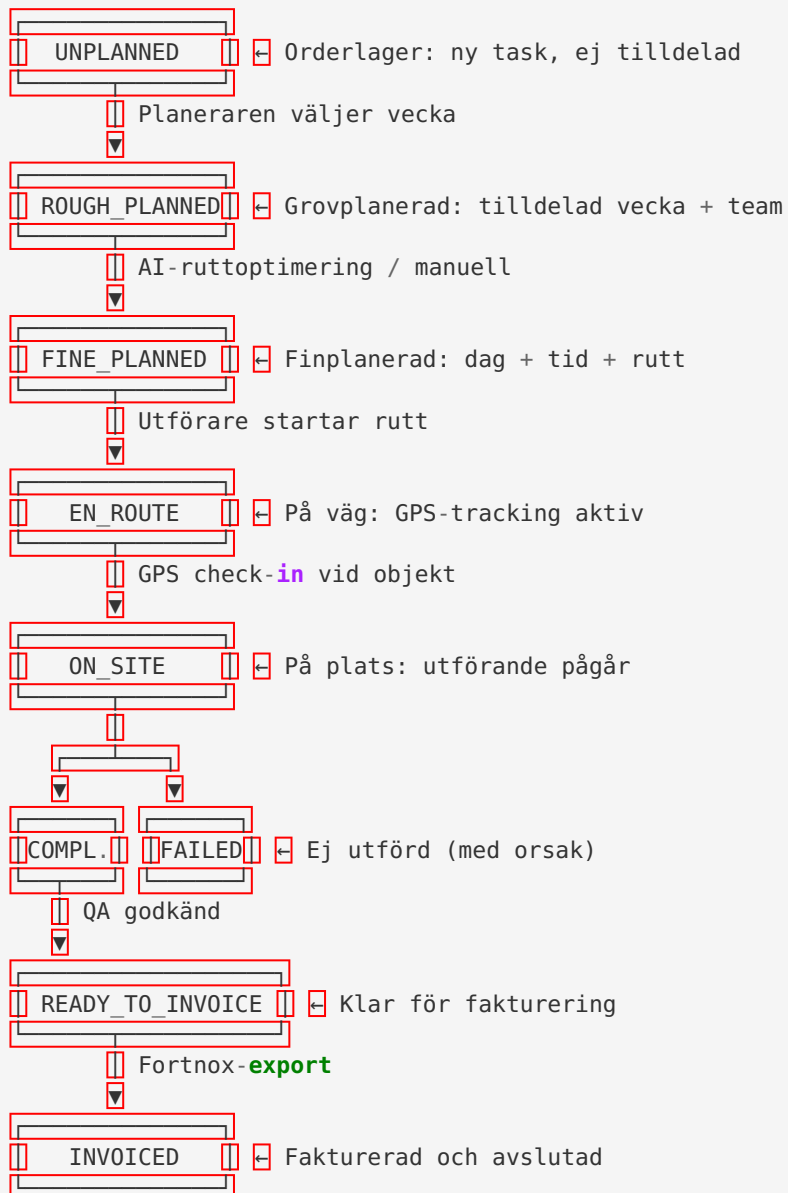
```
// — STEG 7: BEROENDEARTIKLAR → TASK DEPENDENCIES —
for (const task of tasks) {
  const articleDeps = await db.getArticleDependencies(task.articleId);

  for (const dep of articleDeps) {
    const depKey = `${dep.dependsOnArticleId}:${task.serviceObjectId}`;
    const depTask = taskMap.get(depKey);

    if (depTask) {
      dependencies.push({
        workOrderId: `${task.articleId}:${task.serviceObjectId}`,
        dependsOnWorkOrderId: depKey,
        dependencyType: dep.dependencyType,
      });
    }
  }
}

return { tasks, dependencies };
}
```

7. STATUSFLÖDE — 8 Steg



Statusövergångar och validering

Från	Till	Validering	Vem
UNPLANNED	ROUGH_PLANNED	team_id och rough_planned_week satt	Planerare
ROUGH_PLANNED	FINE_PLANNED	scheduled_date + scheduled_time satt	Planerare/AI
FINE_PLANNED	EN_ROUTE	Alla must_complete_first -beroenden uppfyllda	Utförare (Go)
EN_ROUTE	ON_SITE	GPS inom radie från objektets position	Utförare (Go)
ON_SITE	COMPLETED	Checklista klar, data skriven, foto (om krävs)	Utförare (Go)
ON_SITE	FAILED	not_completed_reason ifylld	Utförare (Go)
COMPLETED	READY_TO_INVOICE	QA-check godkänd (kan vara automatisk)	System/Admin
READY_TO_INVOICE	INVOICED	Fortnox-export lyckad, invoiced_at satt	System

Kritisk validering vid EN_ROUTE

```

async function canStartRoute(workOrderId: string): Promise<{
  canStart: boolean;
  blockers: string[];
}> {
  const deps = await db.getWorkOrderDependencies(workOrderId);
  const blockers: string[] = [];

  for (const dep of deps) {
    if (dep.dependencyType === 'must_complete_first') {
      const depWo = await db.getWorkOrder(dep.dependsOnWorkOrderId);
      if (!['completed', 'ready_to_invoice', 'invoiced'].includes(depWo.status)) {
        blockers.push(
          `Beroende "${depWo.article.name}" måste vara klar först (nuvarande: ${dep-
Wo.status})`
        );
      }
    }
  }

  return { canStart: blockers.length === 0, blockers };
}

```

8. PLANERING & KAPACITET

8.1 Orderlager-vy

Planerarens primära arbetslista — alla oplanerade uppgifter.

Filter:

- Utförandetyp (execution_type)
- Geografiskt område / postnummer
- Association / artikeltyp
- Tidsperiod (leveransfönster)
- Kundgrupp

Kvantifiering:

```

-- Orderlageröversikt
SELECT
  a.execution_type,
  COUNT(*) AS task_count,
  SUM(wo.total_time) / 60.0 AS total_hours,
  SUM(wo.total_price) AS total_value
FROM work_orders wo
JOIN articles a ON a.id = wo.article_id
WHERE wo.status = 'unplanned'
  AND wo.tenant_id = :tenant_id
GROUP BY a.execution_type;

```

8.2 Grovplanering (veckoplanering)

- Planeraren väljer tasks från orderlager
- Tilldelar vecka + team (drag-drop)

- Systemet visar kapacitetssummering per vecka/team

8.3 Finplanering (AI-ruttopptimering)

- Befintlig VRP-motor (`route-optimizer.ts`) optimerar rutt
- Tar hänsyn till: geografi, tidsfönster, utförandetyp, restid
- Output: dag + tid + ordning

8.4 Kapacitetsmål: 28/12-regeln

Kinab:s målmodell per 40h arbetsvecka:

Typ	Mål	Beskrivning
Produktionstid	28h	Faktiskt utförande av tjänster
Restid	12h	Transport mellan objekt
Total	40h	Full arbetsvecka

8.5 Kapacitetssummering per vecka/team

```
interface CapacitySummary {
  teamId: string;
  weekNumber: number;
  scheduledProductionHours: number;
  estimatedTravelHours: number;
  taskCount: number;
  utilizationPercent: number; // vs 28h target
  availableHours: number;    // 28 - scheduledProductionHours
}

async function getWeeklyCapacity(
  teamId: string,
  weekStart: Date
): Promise<CapacitySummary> {
  const weekEnd = addDays(weekStart, 7);
  const tasks = await db.getTasksForTeamInRange(teamId, weekStart, weekEnd);

  const totalProdMinutes = tasks.reduce((sum, t) => sum + t.total_time, 0);
  const prodHours = totalProdMinutes / 60;
  const target = (await db.getTeam(teamId)).productionHoursTarget;

  return {
    teamId,
    weekNumber: getWeekNumber(weekStart),
    scheduledProductionHours: prodHours,
    estimatedTravelHours: 0, // Beräknas via VRP
    taskCount: tasks.length,
    utilizationPercent: (prodHours / target) * 100,
    availableHours: target - prodHours,
  };
}
```

8.6 Sökmönster och distriktsfilter

Planeraren skapar sökmönster (statiska eller tillfälliga):

```
{
  "name": "Sundsvall kärltvätt maj",
  "filter_criteria": {
    "execution_types": ["vehicle_team"],
    "postal_codes": ["85230", "85231", "85232"],
    "article_associations": ["RBK Standard"],
    "status": ["unplanned"],
    "date_range": { "from": "2026-05-01", "to": "2026-05-31" }
  },
  "team_id": "{sundsvall_team_id}"
}
```

9. PERIODICITET & FAKTURERING

9.1 Periodicitet

Typ	Intervall	Tolerans	Exempel
WEEKLY	7 dagar	±1 dag	Kontorstädning
MONTHLY	30 dagar	±7 dagar	Kärltvätt
QUARTERLY	90 dagar	±14 dagar	Rumstvätt
BIMONTHLY	60 dagar	±7 dagar	“Varannan månad ±1 vecka”

Periodicitet sätts på OrderConcept:

```
UPDATE order_concepts SET
  is_recurring = TRUE,
  recurrence_interval = 2,
  recurrence_unit = 'months',
  tolerance_days = 7,
  next_generation_date = '2026-07-01'
WHERE id = :concept_id;
```

9.2 Auto-expansion via daemon

```
// Kör dagligen som cron-jobb
async function periodicityDaemon() {
  const now = new Date();

  const dueConcepts = await db.query(`
    SELECT * FROM order_concepts
    WHERE is_recurring = TRUE
      AND concept_status = 'active'
      AND next_generation_date <= $1
  `, [now]);

  for (const concept of dueConcepts) {
    // Idempotency-check
    const alreadyGenerated = await db.query(`
      SELECT 1 FROM work_orders
      WHERE order_concept_id = $1
        AND created_at > $2
      LIMIT 1
    `, [concept.id, concept.lastGeneratedAt]);

    if (alreadyGenerated.length > 0) continue;

    const { tasks } = await expandOrderConcept(
      concept.id,
      concept.nextGenerationDate
    );

    await db.insertTasks(tasks);

    // Beräkna nästa generation
    const nextDate = addInterval(
      concept.nextGenerationDate,
      concept.recurrenceInterval,
      concept.recurrenceUnit
    );

    await db.query(`
      UPDATE order_concepts SET
        last_generated_at = NOW(),
        next_generation_date = $1
      WHERE id = $2
    `, [nextDate, concept.id]);
  }
}
```

9.3 Månadsfakturerering

Vid `billing_mode = 'monthly'`:

Årsvärde = 120 000 kr
 Månadsfaktura = 120 000 / 12 = 10 000 kr

Vid ändring (t.ex. antal käll 10→12 den 15 maj):

Maj: $(10\,000 \times 15/31) + (12\,000 \times 16/31) = 4\,838 + 6\,194 = 11\,032$ kr
 Juni-dec: 12 000 kr/månad

9.4 Indexjustering

```

async function applyIndexAdjustment(
  priceListId: string,
  adjustmentPercent: number, // t.ex. 2.0 = +2%
  effectiveDate: Date
) {
  const articles = await db.getArticlesByPriceList(priceListId);

  for (const article of articles) {
    const newPrice = article.price * (1 + adjustmentPercent / 100);
    const newCost = article.cost * (1 + adjustmentPercent / 100);

    await db.updateArticle(article.id, {
      price: Math.round(newPrice * 100) / 100,
      cost: Math.round(newCost * 100) / 100,
    });

    // Logga indexändring
    await db.insertPriceHistory({
      articleId: article.id,
      oldPrice: article.price,
      newPrice,
      reason: `Indexjustering +${adjustmentPercent}%`,
      effectiveDate,
    });
  }

  // Omräkna framtida månadsfakturer
  await recalculateMonthlyInvoices(priceListId, effectiveDate);
}

```

9.5 Retroaktiv justering

Vid metadata-ändring (t.ex. antal käll ändras):

1. Arkivera gamla metadata (version $N \rightarrow N+1$)
2. Beräkna om årsvärde för berörda orderkoncept
3. Justera framtida månadsfakturer från ändringsdatum
4. Logga ändring med full spårbarhet

9.6 Export till Fortnox

```

async function exportToFortnox(workOrderIds: string[]) {
  const workOrders = await db.getWorkOrders(workOrderIds);

  // Gruppera per kund
  const byCustomer = groupBy(workOrders, wo => wo.customerId);

  for (const [customerId, wos] of byCustomer) {
    const invoice = {
      CustomerNumber: customer.fortnoxId,
      InvoiceRows: wos.map(wo => ({
        ArticleNumber: wo.article.articleNumber,
        Description: wo.article.invoiceDescription,
        DeliveredQuantity: wo.frozenQuantity,
        Price: wo.frozenUnitPrice,
        Unit: wo.frozenUnit,
      })),
    };

    await fortnoxClient.createInvoice(invoice);

    // Markera som fakturerade
    for (const wo of wos) {
      await db.updateWorkOrderStatus(wo.id, 'invoiced', {
        invoicedAt: new Date()
      });
    }
  }
}

```

10. MOBIL-APP INTEGRATION (Go)

Vad Go-appen behöver från backend

Endpoint	Data	Syfte
GET /api/v1/mobile/tasks	Dagens tasks för teamet	Arbetslista
GET /api/v1/mobile/tasks/:id	Task-detalj med checklista	Utförande
GET /api/v1/mobile/tasks/:id/metadata	Metadata-snapshot	Data read
PATCH /api/v1/mobile/tasks/:id/status	Statusuppdatering	Check-in/out
POST /api/v1/mobile/tasks/:id/result	Resultat + metadata write	Data write
POST /api/v1/mobile/tasks/:id/photo	Foto-upload	Dokumentation

Checklista från strukturartiklar (BOM)

```
// GET /api/v1/mobile/tasks/{parent_id}/checklist
{
  "parentTask": {
    "id": "...",
    "article": "TILG100 Kodlås",
    "object": "Soprum Södra, Butik Malmö"
  },
  "checklistItems": [
    {
      "order": 1,
      "taskId": "...",
      "article": "TILG201 Kontrollera kod",
      "status": "on_site",
      "dataRead": { "KOD": "4521" },
      "dataWrite": { "KOD": null },
      "format": "numeric"
    },
    {
      "order": 2,
      "taskId": "...",
      "article": "TILG202 Öppna med kod",
      "status": "fine_planned",
      "dataRead": { "KOD": "4521" }
    }
  ]
}
```

Data read/write (snapshot → result)

- Vid expansion: metadata-snapshot sparas på work_order
 ➡ work_order.metadata_snapshot = { "ANT": 10, "KOD": "4521" }
- Utförare ser snapshot i appen
- Utförare uppdaterar (ANT: 10 ➡ 12)
 ➡ POST /api/v1/mobile/tasks/:id/result
 ➡ body: { "data_writes": { "ANT": 12 } }
- Backend:
 - Arkiverar gamla metadata ANT=10
 - Skapar ny metadata ANT=12 (method="at_execution")
 - Sparar result_data på work_order

GPS check-in/check-out

```
// PATCH /api/v1/mobile/tasks/:id/status
{
  "status": "on_site",
  "latitude": 59.3293,
  "longitude": 18.0686
}

// Backend validerar avstånd till objektets position
const distance = haversine(
  { lat: request.latitude, lng: request.longitude },
  { lat: object.metadata.GPS.lat, lng: object.metadata.GPS.lng }
);

if (distance > MAX_CHECKIN_RADIUS_METERS) {
  throw new Error(`GPS-position ${distance}m från objektet (max ${MAX_CHECKIN_RADIUS_M
ETERS}m)`);
}
```

Offline-support

Go-appen måste hantera offline:

- Cacha dagens tasks + metadata-snapshots lokalt
- Köa statusuppdateringar, foton och data writes
- Synca vid åter-anslutning (conflict resolution: server wins på metadata, append på foton)

11. IMPLEMENTERINGSPLAN — 7-8 Sprints

Sprint 1: Metadata-subsystem (5 dagar)

Uppgift	Estimat
DDL: service_objects (minimal)	0.5 dag
DDL: object_metadata (EAV med arv)	1 dag
get_effective_metadata() SQL-funktion	1 dag
CRUD storage-funktioner	1 dag
API-routes + tester	1.5 dag

Leverans: Metadata-subsystem med arv, nivååls och versionering.

Sprint 2: Artiklar & Struktur (4 dagar)

Uppgift	Estimat
DDL: <code>articles</code> (alla 33 fält)	0.5 dag
DDL: <code>article_components</code> + <code>article_dependencies</code>	0.5 dag
<code>article_translations</code>	0.25 dag
CRUD storage + API	1.5 dag
BOM-hantering + dependency-validering	1.25 dag

Leverans: Kompletta artikelmaster med BOM och beroenden.

Sprint 3a: Expansion & Dependencies (5 dagar)

Uppgift	Estimat
DDL: <code>order_concepts</code> + <code>order_concept_lines</code>	0.5 dag
DDL: <code>work_orders</code> (utökad) + <code>work_order_dependencies</code>	0.5 dag
<code>expandOrderConcept()</code> — full implementation	2 dagar
Dependency-koppling + validering	1 dag
Tester med Örebro Bostäder-data	1 dag

Leverans: Expansion från koncept till tasks med BOM och beroenden.

Sprint 3b: Planering & Kapacitet (5 dagar)

Uppgift	Estimat
DDL: <code>teams</code> + <code>planner_search_filters</code>	0.5 dag
Orderlager-API (filter, summering)	1.5 dag
Kapacitetssummering per vecka/team	1 dag
Sökmönster CRUD + filter-logik	1 dag
Integration med befintlig planeringvy	1 dag

Leverans: Orderlager, sökmönster och kapacitetsöversikt.

Sprint 4: Statusflöde (3 dagar)

Uppgift	Estimat
Status-enum + tidsstämplar	0.5 dag
Statusövergångsvalidering	1 dag
Dependency-check vid EN_ROUTE	0.5 dag
Larm vid ej uppfyllt beroende	1 dag

Leverans: 8-steps statusflöde med validering och larm.

Sprint 5: Orderlager UI (8 dagar)

Uppgift	Estimat
Orderlager-vy (lista, filter, summering)	3 dagar
Grovplaneringsvy (vecko-tilldelning, drag-drop)	3 dagar
Kapacitetsvy (visuell summering)	2 dagar

Leverans: Kompletta planeringsgränssnitt.

Sprint 6: Periodicitet & Fakturering (5 dagar)

Uppgift	Estimat
Periodicitetsdaemon (cron)	1 dag
Toleranshantering	0.5 dag
Månadsfakturerings med omräkning	1.5 dag
Indexjustering	1 dag
Fortnox-exportintegration	1 dag

Leverans: Automatisk återgenerering, faktureringsmodeller, Fortnox-export.

Sprint 7: Pop-out & Advanced (7 dagar)

Uppgift	Estimat
Koncept-hanteringsvy (skapa, redigera, pre-view)	2 dagar
Artikelmaster-editor (33 fält)	2 dagar
Metadata-hanteringsvy med arv-visualisering	2 dagar
AI-varningar (regelbaserade)	1 dag

Leverans: Administrationsgränssnitt för koncept, artiklar och metadata.

Sprint 8: Go-integration & Testing (5 dagar)

Uppgift	Estimat
Mobile API-endpoints	1.5 dag
Checklista-API (BOM)	0.5 dag
Data read/write API	1 dag
End-to-end-test hela flödet	2 dagar

Leverans: Komplet Go-appintegration och systemtest.

Totalöversikt

Sprint	Dagar	Veckor
Sprint 1: Metadata	5	1
Sprint 2: Artiklar	4	~1
Sprint 3a: Expansion	5	1
Sprint 3b: Planering	5	1
Sprint 4: Statusflöde	3	~0.5
Sprint 5: Orderlager UI	8	~1.5
Sprint 6: Periodicitet	5	1
Sprint 7: Advanced UI	7	~1.5
Sprint 8: Go + Test	5	1
TOTALT	~47 dagar	~9-10 veckor

⚠ ROM-estimat. Lägg till ~20% buffert → **~11-12 veckor** realistiskt.

12. DATAMIGRATION

Import från Excel

```

Steg 1: Prislsta.xlsx → articles
Steg 2: Artikel.xlsx → articles (utökad) + article_components
Steg 3: OBJEKT.xlsx → service_objects + object_metadata
Steg 4: Definiera → order_concepts + order_concept_lines
Steg 5: Utförda uppgifter.xlsx → work_orders (historiska, status='completed')

```

Mapping: Prislista.xlsx → articles

Excel-kolumn	DB-fält	Transformation
Artikelnummer	article_number	Direkt
Beskrivning	name + invoice_description	Direkt
Assosiation	association	Direkt (NULL för AV100)
Kostnad	cost	Numerisk
Pris	price	Numerisk
Utförande tid	production_time	Numerisk (minuter)
Ofsettid	offset_hours	Evaluera formel (=40*60 → 2400 min → 40h)

Mapping: OBJEKT.xlsx → service_objects + object_metadata

```
def import_objects(df, customer_id, tenant_id):
    # 1. Skapa toppnod (Område)
    area = insert_service_object(
        name=df.iloc[0]['Namn'], # "Örebrobostäder"
        customer_id=customer_id,
        parent_id=None
    )

    # 2. Gruppera per adress → skapa mellanobjekt (plats)
    for address, group in df.iloc[1:].groupby('Gata'):
        place = insert_service_object(
            name=address,
            customer_id=customer_id,
            parent_id=area.id
        )
        # Adress-metadata på platsnivå (ärvs nedåt)
        insert_metadata(place.id, 'Adress', address, level_locked=False)
        insert_metadata(place.id, 'Postnummer', group.iloc[0]['Postnummer'],
            level_locked=False)

    # 3. Skapa objekt under varje plats
    for _, row in group.iterrows():
        obj = insert_service_object(
            name=row['Namn'],
            customer_id=customer_id,
            parent_id=place.id
        )
        # Association (nivålåst – exakt detta objekt)
        insert_metadata(obj.id, 'Association', row['Värde.1'], level_locked=True)
        # Kvantitet (nivålåst)
        insert_metadata(obj.id, 'ANT', float(row['Värde.2'] or 0), level_locked=True)

    # Objekttyp
    insert_metadata(obj.id, 'Objekttyp', row['Värde'], level_locked=True)
```

Befintliga 3 750+ work_orders från Kinab

- `order_concept_id = NULL` (skapade före konceptmodellen)
- Ingen migration krävs — befintliga work_orders opåverkade
- Kan retroaktivt grupperas till koncept om affärsnytta finns

Datakvalitetsproblem

Problem	Antal	Hantering
Barnobjekt saknar objekt-ID	1 164	Generera UUID, logga mapping
Adressmismatch (versaler/stavning)	15	Normalisera + manuell review
Association <code>a_istf</code> <code>RBK Standard</code>	1	Flagga som datakvalitetsfel
<code>AntalKar1</code> används för m ²	1 938	Mappa till generiskt <code>quantity</code>
Noll-kvantiteter	1 174	Importera men flagga
Dubbelrader	Minst 1	Dedup-logik
Snapshot-drift (master ≠ historik)	Okänt	Frys kvantitet på work_order
Excel-formler i offsettid	N100	Evaluera $=40*60 \rightarrow 2400$

13. ÖPPNA FRÅGOR & BESLUT

Från metadata/artikel-sessionen (1-7)

#	Fråga	Status
1	Plats som metadata eller fast fält?	BESLUTAT: Dubbellagring — metadata (arvbar) + denormaliserad för VRP
2	Metadatabeteckningars namnkonvention	REKOMMENDATION: <code>metadata_types</code> -tabell (jfr kontoplan)
3	Objekt på flera platser?	BESLUTAT: Varna, inte blockera
4	Leveranstid för tjänster ("Ställ/resa")	ÖPPEN: Beräkna dynamiskt via VRP
5	Subtyper av tjänster	REKOMMENDATION: <code>execution_type</code> hanterar detta, inte <code>article_type</code>
6	Team/projektnummer på metadata	BESLUTAT: Hör hemma på <code>work_orders/teams</code> , ej metadata
7	En uppgiftsrad = en data-operation	BEKRÄFTAD: <code>data_read_keys</code> + <code>data_write_keys</code> per artikel

Från planeringssessionen (8-13)

#	Fråga	Beskrivning
8	Periodicitetstolerans — vem bestämmer exakt datum?	Planeraren eller systemet? Optimera baserat på kapacitet?
9	Månadsfakturerering — retroaktiv justering?	Redan fakturerade månader: justera eller bara framtida?
10	Sökmönster — teknisk scope	Delning, schemaläggning, geografisk precision?
11	Vilohantering — regelverket	Arbetstidslagen, automatisk blockering?
12	Automatiska uppgifter — vilka i v1?	Aviseringar? Tidpunkt? Automarkering?
13	Inställdesresor — arbetstid?	Flagga per resa? Påverkar kapacitetsmålet?

14. KONKRETA EXEMPEL

“Hästtvätt 26” — från OrderConcept till 13 tasks

Scenario: Kinab utför kärltvätt för Axfood, Butik Malmö. Soprummet har 10 matavfallskärl + kodlås dörr. Nyckel måste hämtas.

OrderConcept: “Axfood Hösttvätt 2026”

Artiklar i konceptet:

1. N100 ☐ Hämta nyckel (offset: 40h, single)
2. TILG100 ☐ Kodlås (struktur: TILG201 + TILG202, single)
3. A100 ☐ Telefonavisering (offset: 2h, single)
4. RBK100 ☐ Kärltvätt (multiply ☒ ANT)
5. F100 ☐ Fotodokumentation (single)

Expansion mot objekt “Matavfallskärl Malmö” (ANT=10, Association=RBK1):

#	Task	Artikel	Qty	Pris	Tid	Sched-uled	Ber-oenden
1	Hämta nyckel	N100	1	0 kr	0 min	-40h	—
2	Tele-fonav-isering	A100	1	0 kr	1 min	-2h	—
3	Kontrol-lera kod	TILG201	1	0 kr	5 min	-20 min	—
4	Öppna med kod	TILG202	1	0 kr	1 min	huvudtid	#3
5	Kärلتvätt ×10	RBK100	10	2 500 kr	30 min	huvudtid	#1, #3
6	Fotodok-umenta-tion	F100	1	0 kr	0.5 min	huvudtid	#5

TILG100 expanderar till checklista (sub-tasks under parent):

Task #3 (parent: TILG100):

- └ Checklist item 1: TILG201 "Kontrollera kod" – Hämta KOD=4521, Lämna ny KOD
- └ Checklist item 2: TILG202 "Öppna med kod" – Hämta KOD=4521, Lämna inget

Task #5 (RBK100 Kärلتvätt) metadata-flöde:

1. Expansion: snapshot ANT=10 från objektmetadata
2. Utförare anländer, räknar 12 kärl
3. Utförare rapporterar: actual_quantity=12, data write: ANT=12
4. Backend: arkiverar ANT=10, skapar ANT=12 (method=at_execution)
5. Fakturering: 12 x 250 kr = 3 000 kr (inte 2 500 kr)

Tidslinje om huvudjobb = Måndag 09:00:

Torsdag 17:00 → N100: Hämta nyckel (40h före)
 Måndag 07:00 → A100: Ring kunden (2h före)
 Måndag 08:40 → TILG201: Kontrollera kod (20 min före)
 Måndag 09:00 → TILG202: Öppna dörr
 Måndag 09:00 → RBK100: Tvätta 10 kärl (30 min)
 Måndag 09:30 → F100: Fotografera resultat

15. API ENDPOINTS

Orderkoncept

POST	/api/v1/order-concepts	Skapa nytt koncept
GET	/api/v1/order-concepts	Lista koncept (med filter)
GET	/api/v1/order-concepts/:id	Hämta koncept med linjer
PUT	/api/v1/order-concepts/:id	Uppdatera koncept
POST	/api/v1/order-concepts/:id/expand	Expandera till tasks
POST	/api/v1/order-concepts/:id/preview	Preview expansion (utan att skapa)
DELETE	/api/v1/order-concepts/:id	Arkivera koncept

Orderlager & Planering

GET	/api/v1/orderlager/tasks	Lista oplanerade tasks (med filter)
GET	/api/v1/orderlager/summary	Sammanfattning per typ/team
GET	/api/v1/capacity/:teamId/week/:weekNum	Kapacitetsöversikt
POST	/api/v1/planning/rough	Grovplanera (tilldela vecka/team)
POST	/api/v1/planning/fine	Finplanera (tilldela dag/tid/rutt)

Tasks

PATCH	/api/v1/tasks/:id/status	Statusövergång
GET	/api/v1/tasks/:id/dependencies	Lista beroenden
GET	/api/v1/tasks/:id/checklist	BOM-checklista
POST	/api/v1/tasks/:id/result	Rapportera resultat + data write

Metadata

GET	/api/v1/objects/:id/metadata	Alla metadata (inkl ärvda)
GET	/api/v1/objects/:id/metadata/effective	get_effective_metadata()
POST	/api/v1/objects/:id/metadata	Skapa metadata
PUT	/api/v1/metadata/:id	Uppdatera (skapar ny version)
GET	/api/v1/metadata/:id/history	Versionshistorik

Artiklar

GET	/api/v1/articles	Lista artiklar
POST	/api/v1/articles	Skapa artikel
PUT	/api/v1/articles/:id	Uppdatera
GET	/api/v1/articles/:id/components	BOM-komponenter
GET	/api/v1/articles/:id/dependencies	Beroenden

Sökmönster

GET	/api/v1/search-filters	Lista sökmönster
POST	/api/v1/search-filters	Skapa sökmönster
DELETE	/api/v1/search-filters/:id	Ta bort

Fakturerering

POST	/api/v1/invoicing/ export	Exportera till Fortnox
GET	/api/v1/invoicing/monthly-preview	Preview månadsfaktura
POST	/api/v1/invoicing/index-adjustment	Applitera indexjustering

Mobile (Go)

GET	/api/v1/mobile/tasks	Dagens tasks för team
GET	/api/v1/mobile/tasks/:id	Task-detalj + checklista
GET	/api/v1/mobile/tasks/:id/metadata	Metadata-snapshot
PATCH	/api/v1/mobile/tasks/:id/status	Statusuppdatering + GPS
POST	/api/v1/mobile/tasks/:id/result	Resultat + data write
POST	/api/v1/mobile/tasks/:id/photo	Foto-upload
POST	/api/v1/mobile/sync	Batch sync (offline-kö)

16. TEKNISKA CONSTRAINTS

Bakåtkompatibilitet

- Befintliga 3 750+ work_orders opåverkade (order_concept_id = NULL)
- Mobile Go-app ser bara work_orders — ingen ändring krävs
- VRP/AI-planner oförändrad — execution_code cachat på work_order
- Fortnox-export oförändrad — fakturerar mot work_orders + lines
- Modus-import oförändrad — skapar work_orders direkt

Performance (3 750+ orders)

```
-- Kritiska index
CREATE INDEX idx_wo_status ON work_orders(status);
CREATE INDEX idx_wo_team_date ON work_orders(team_id, scheduled_date);
CREATE INDEX idx_wo_concept ON work_orders(order_concept_id);
CREATE INDEX idx_om_active ON object_metadata(object_id, status) WHERE status = 'active';

-- get_effective_metadata() – använder CTE med RECURSIVE
-- Begränsa djup: max 10 nivåer (LIMIT i recursive CTE)
-- Cacha resultat vid expansion (metadata_snapshot på work_order)
```

Offline-support (Go)

- Cacha alla tasks + metadata lokalt vid sync
- Köa alla mutations lokalt
- Batch sync vid connectivity
- Conflict resolution: server wins på metadata, merge på tasks
- Foto-upload kan ske async

Transaktionskrav (bokföringslagen)

- **Fullständig audit trail** — alla ändringar loggas
- **Momskoder och kontering** på artiklar
- **Fakturor kan inte raderas** — enbart krediteras

- **Spårbarhet** — vem, när, vad för alla transaktioner
- `work_orders.invoiced_at` och `invoiced_at` är immutabla efter sättnig

Säkerhet (row-level security)

```
-- Alla tabeller har tenant_id
-- RLS-policy:
ALTER TABLE work_orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON work_orders
    USING (tenant_id = current_setting('app.current_tenant_id')::UUID);

-- Samma mönster för alla tabeller
```

Detta dokument är den definitiva implementeringsguiden för Traivo.
Genererat 2026-05-06 baserat på alla JSON-sessionsanalyser och Excel-filer.
Kopiera in i Replit och använd som single source of truth.